

B28: SQL SuperHero Program : Session 10

1 message

Vishal Jaiswal <vishaldbdev@gmail.com>
Bcc: satishhattekar1997@gmail.com

28 February 2025 at 08:27

Having Clause :

Having --> having is just like a where clause on group by. The HAVING clause was introduced in SQL to allow the filtering of query results based on aggregate functions and groupings, which cannot be achieved using the WHERE clause that is used to filter individual rows.

```

1 SELECT Region_Id, Count(Country_Name)
2 FROM Countries
3 group by Region_Id
4 having Count(Country_Name) > 5;
5
6 --> i want all region who have more than 5 countries
7 --> count all countries in each region
8 --> then take only those who have more than 5
9

```

Script Output x Query Result x Query Result 1 x

SQL | All Rows Fetched: 3 in 0.604 seconds

REGION_ID	COUNT_COUNTRY
1	8
2	6
3	6

```
select count(*), department_id from hr.employees group by department_id
```

COUNT(*)	DEPARTMENT_ID
45	50
1	40
2	110
3	90
6	30
1	70
1	-
1	10
2	20
5	60
6	100


 All employees count
for each
department

But the question is : I do not want all departments, only those in which departments have more than 10 employees. How do I know?

First count and then only we can say that this department has more than 10 employees. so whenever we have to apply filters after getting count, max, min, sum and avg it means having.

```
SELECT COUNT(*), DEPARTMENT_ID FROM HR.EMPLOYEES GROUP BY DEPARTMENT_ID HAVING COUNT(*) > 10
```

COUNT(*)	DEPARTMENT_ID
45	50
34	80

2 rows selected.

? → only 2
Satisfy the condition

Filter on aggregate data

```
select count(*), department_id from hr.employees group by department_id where count(*) > 10
```

ORA-00933: SQL command not properly ended

where clause can not filter on aggregate

```
select count(*), department_id from hr.employees group by department_id having count(*) > 10
```

COUNT(*)	DEPARTMENT_ID
45	50
34	80

for filter data on aggregate f^h

Can we write Having clause before group by in sql query?

YES. There is no difference in behavior. It works the same.

```
select department_id, count(*) from hr.employees group by department_id having count(*) > 5
```

DEPARTMENT_ID	COUNT(*)
50	45
30	6
100	6
80	34

4 rows selected.

```
select department_id, count(*) from hr.employees having count(*) > 5 group by department_id
```

DEPARTMENT_ID	COUNT(*)
50	45
30	6
100	6
80	34

4 rows selected.

Same

Having clause without Group By

Having is applied after the aggregation phase and must be used if you want to filter aggregate results. So the reverse isn't true

When GROUP BY is not used, the WHERE and HAVING clauses are essentially equivalent.

However, when GROUP BY is used:

The WHERE clause is used to filter records from a result. The filtering occurs before any groupings are made.

The HAVING clause is used to filter values from a group (i.e., to check conditions after aggregation into groups has been performed).

- WHERE clause filters individual rows, whereas the HAVING clause filters groups instead of one row at a time.
- We cannot use the WHERE clause with aggregate functions because it works for filtering individual rows. In contrast, HAVING can work with aggregate functions because it is used to filter groups.

```
select max(salary) from hr.employees having max(salary) > 10000
```

MAX(SALARY)
24000

used without group by

Behave like where

```
select max(salary) from hr.employees having max(salary) > 30000
```

no data found

UPPER or LOWER keywords not used in both places in select expression and group by expression . That's why 2nd statement will not run and it gives an error. Whatever we select, we have to group by the same thing.

In Group By, exact same thing will be copied which is selected along with max, min, sum, count, avg.

```
select department_id , count(*) from hr.employees group by upper(department_id)
```

ORA-00979: not a GROUP BY expression

group by is exact same
as in select

```
select upper(department_id) , count(*) from hr.employees group by upper(department_id)
```

UPPER(DEPARTMENT_ID)	COUNT(*)
90	3

Query order of execution

1. FROM and JOINS

The FROM clause, and subsequent JOINS are first executed to determine the total working set of data that is being queried.

2. WHERE

Once we have the total working set of data, the first-pass WHERE constraints are applied to filter data.

3. GROUP BY

The remaining rows after the WHERE constraints are applied are then grouped based on common values in the column specified in the GROUP BY clause.

4. HAVING

filter data from aggregate

5. SELECT

Any expressions in the SELECT part of the query are finally computed.

6. DISTINCT

Of the remaining rows, rows with duplicate values in the column marked as DISTINCT will be discarded.

7. ORDER BY

If an order is specified by the ORDER BY clause, the rows are then sorted by the specified data in either ascending or descending order. Since all the expressions in the SELECT part of the query have been computed, you can reference aliases in this clause.

8. LIMIT / OFFSET/ fetch/ rownum

(We will discuss on this later)

INTERSECT : Gives us common

test 1 has 1,2,3,4

test2 has 2,3,4,5

```
select * from test12
intersect
select * from test11;
```

```
select * from test1
intersect
select * from test2
```

ID
2
3
4

} Common

3 rows selected.

it return 2,3,4 --> because these values are common in both tables

MINUS: (SQL SERVER IT IS EXCEPT)

a-b --> what is in A and Not in B

b-a -->. what is in B and not in A

```
select * from test1
```

ID
1
3
2
7

4 rows selected.

```
select * from test2
```

ID
1
4
2
6

4 rows selected.

```
select * from test2  
minus  
select * from test1
```

ID
4
6

2 rows selected.

4,6 are in test2,
but not in test1

-> it gives 4,6 , because these are exists in test2 but not in test1

```
select * from test1  
minus  
select * from test2;
```

```
select * from test1  
minus  
select * from test2
```

ID
3
7

2 rows selected.

--> it gives us 3,7 because these are exists in test1 not in test2;

Lets Insert a Null record in both tables. Remember Distinct, UNION, Intersect and Minus will able to match null with null.

```
insert into test1 values (null)
```

1 row(s) inserted.

```
insert into test2 values (null)
```

1 row(s) inserted.

Both table inserted
Null
[Minus will recognize]

```
select * from test1
minus
select * from test2
```

ID
3
7

2 rows selected.

→ only 3,7 coming
bcoz it recognize
'null' presents both
tables.

```
delete from test2 where id is null
```

1 row(s) deleted.

→ let's remove null from
test2 table

```
select * from test1
minus
select * from test2
```

ID
3
7
-

3 rows selected.

→ 3,7, null
these 3 are in test 1
& not in test 2